

第2章 程序设计基础知识

对应《信息学奥赛一本通·初赛篇》第二章 | 授课时长：约 50 分钟
适用范围：CSP-J / NOIP 普及组初赛

 本节将前 7 节合为一次课，第 8 节树、第 9 节图后续单独安排。

[[pcbc/C++课程/CSP/课件/第二程序设计基础/习题1老师|习题1老师]]
[[pcbc/C++课程/CSP/课件/第二程序设计基础/习题2老师|习题2老师]]
[[习题3老师(较难-链表加强)]]

第一节 程序基本常识（约 8 min）

1.1 算法的五大特征

特征	含义	反例
有穷性	必须在有限步内结束	死循环
确定性	每步指令唯一确定、无歧义	模棱两可的描述
可行性	每一步都能在有限时间内完成	理论上可行但实际算不了
输入	有 0 个或多个输入	—
输出	有至少 1 个输出	没有结果

★ 初赛常考：给一段描述判断是否符合算法特征，尤其爱考「有穷性」和「确定性」。

1.2 时间复杂度

算法执行时间随数据规模 n 增长的变化趋势，用 **大 O 表示法** 描述。

常见复杂度排序（由快到慢）：

1 | $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

计算规则：

- 忽略常数： $O(3n) = O(n)$
- 只取最高阶： $O(n^2 + n) = O(n^2)$
- 乘法规则： 嵌套循环 = 各层复杂度相乘
- 加法规则： 顺序执行的代码 = 取复杂度最大的部分

★ 典型代码复杂度：

代码模式	复杂度	说明
单层循环 <code>for(i=0;i<n;i++)</code>	$O(n)$	执行 n 次
双层 <code>for i→n: for j→n</code>	$O(n^2)$	两层各 n 次
双层 <code>for i→n: for j→i</code>	$O(n^2)$	约 $n^2/2$ （内层依赖外层）
三层 <code>for i→n: for j→n: for k→n</code>	$O(n^3)$	三层各 n 次
二分 <code>while(l<r){mid=(l+r)/2}</code>	$O(\log n)$	每次规模减半
递归 <code>fib(n)=fib(n-1)+fib(n-2)</code>	$O(2^n)$	指数爆炸

真题示例（CSP-J 2020）：

- 1 以下哪个时间复杂度描述是错误的？
- 2 A. $O(n + n^2) = O(n^2)$ ← 正确，取最高阶
- 3 B. $O(2n) = O(n)$ ← 正确，忽略常数
- 4 C. $O(n^2 + n \log n) = O(n^2)$ ← 正确
- 5 D. $O(2^n + n^{10}) = O(n^{10})$ ← 错误！指数增长 > 多项式

答案：D

1.3 空间复杂度

- 算法运行过程中所需的额外存储空间
- 常见： $O(1)$ （原地操作）、 $O(n)$ （额外数组）、 $O(n^2)$ （二维数组）
- ★ 递归的空间复杂度 = 递归深度（额外栈空间）

第二节 C++ 语言基础（约 8 min）

本节快速过框架，重点放在类与指针（约 5 min）。

2.1 程序基本构成

```
1 #include <iostream>    // 头文件
2 using namespace std;   // 命名空间
3
4 int main() {           // 主函数
5     int a = 10;
6     cout << a;         // 输出
7     return 0;          // 返回值
8 }
```

- `main` 函数是程序入口，`return 0` 表示正常退出
- `cin` / `cout` 在 `<iostream>` 中定义

2.2 数据类型速览

类型	字节	范围
int	4	$-2^{31} \sim 2^{31}-1$
long long	8	$-2^{63} \sim 2^{63}-1$
float	4	约 $\pm 3.4 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$
double	8	约 $\pm 1.7 \times 10^{-308} \sim \pm 1.7 \times 10^{308}$
char	1	$-128 \sim 127$ / $0 \sim 255$
bool	1	true / false

2.3 程序三种基本结构

结构	C++ 语法
顺序	代码从上往下依次执行
分支	if-else / switch-case
循环	for / while / do-while

★ 初赛常考：for 循环的执行顺序（初始化→条件判断→循环体→迭代→条件判断.....）

2.4 ★ 类与指针（重点 5 min）

类（Class）

```
1 struct people{ // 默认权限为public
2     int id;
3     string name;
```

```

4     void my_name(){
5         cout << name << '\n';
6     }
7 }
8 class Student { // 默认权限为private
9 private:
10     int id;          // 私有成员，仅类内部可访问
11     string name; // 成员属性(成员函数)
12 protected:
13     string school; // 保护成员，类内部和派生类可访问
14 public:
15     void setName(string n) { name = n; } // 公有成员函数，都可以访问
16     void setSchool(string n) { school = n; } // 成员方法(成员函数)
17     string getName() { return name; }
18 };
19 // Stu 是 Student 的子类(派生类)
20 // Student 是 Stu 的父类(基类)
21 // 继承是向下兼容，变成本身权限和继承权限的g
22 class Stu : public Student {
23 public:
24     void printId() {
25         cout << id << '\n'; // 报错，子类无法访问父类的私有属性
26     }
27     void printSchool() {
28         cout << school << '\n'; // 可以访问
29     }
30 };
31
32 Stu test;
33 test.school = "asd"; // 访问不了，只能在类内部访问

```

```
3 test.setSchool("asd"); // 公开的，可以在类外面访问
5 test.printSchool(); // 公开的，可以在类外面访问
```

三种访问权限：

权限	关键字	类内部	派生类	外部
公有	<code>public</code>	✓	✓	✓
保护	<code>protected</code>	✓	✓	✗
私有	<code>private</code>	✓	✗	✗

- `protected` 介于两者之间：自己能用的，派生类能用，**外部不能用**
- **成员函数**是定义在类内部的函数
- ★ 初赛常考：给一段类定义代码，判断哪些访问合法/非法
- 类的默认权限是`private`，结构体的默认权限是`public`

联想string

指针 (Pointer)

```
1 // 什么是指针
2 int a = 1;
3 &a; // &a 取地址，获取a的地址， 地址就是指针 指针就是地址
4 int * p; // 创建一个 指针， 指向 int类型 的 指针
5 p = &a; // p 指向了 a;
6 *p; // *p 就代表 p 指向的内容
7
8 *p = 2;
9 cout << a;
10
11 // 指针与数组
12 int arr[5] = {1,2,3,4,5};
```

```

2  int *q = arr;           // 数组名就是首地址
3  cout << *(q+2);        // → 3
4  cout << sizeof(arr);   // 20 数组内存大小
5
6  // 指针大小
7  int *p1;
8  long long *p2;
9  double *p3;
0  long *p4;
2  指针大小一样大，看操作系统位数，32位是4B, 64位是8B
2  可以通过sizeof关键字验证、可以发现，数组名和指针还是有些区别的。
3
4  // 结构体/类 指针
5  struct stu{
6      int age, height;
7      string name;
8  } a;
9  stu *p = &a;
0
1  (*p).age; // 解引用之后 点访问
2  p->name; // 指针 直接 箭头访问， 通过地址 也可以直接访问成员

```

操作	含义	示例
<code>&x</code>	取地址	<code>int *p = &x;</code>
<code>*p</code>	解引用（取指针指向的值）	<code>cout << *p;</code>
<code>p + k</code>	指针偏移 k 个元素	<code>*(p+2)</code> 等价于 <code>arr[2]</code>
<code>p->a</code>	p所指向的对象的属性	<code>p->name</code>

指针常考陷阱：

1. 未初始化的指针（野指针）

2. `NULL` / `nullptr` 的解引用会崩溃

3. 指针 + 整数是偏移，不是地址 + 字节数；`int *p` 时 `p+1` 跳过 4 字节

2.5 函数

```
1 // 值传递（形参改变不影响实参）
2 void swap1(int a, int b) { int t = a; a = b; b = t; }
3
4 // 址传递（把变量的内存地址传递，会影响变量本身）
5 void swap3(int *pa, int *pb){
6     int t = *pa;
7     *pa = *pb;
8     *pb = t;
9 }
1
0 // 引用传递（形参改变影响实参）
1 void swap2(int &a, int &b) { int t = a; a = b; b = t; }
```

- **值传递**：复制一份，函数内修改不影响外部（★ 常考）
- **引用传递**：直接操作原变量
- **址传递**：传指针，通过指针直接修改内存内容
- 全局变量与局部变量作用域区分、什么是生命周期、什么是作用域

第三节 排序算法（约 5 min）

只需记住核心属性，不解释原理。

常用排序算法对比表

排序算法	平均时间复杂度	最优	最坏	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	✓ 稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ 不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	✓ 稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ 不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ 稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ 不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	✓ 稳定
桶排序	$O(n + k)$	$O(n)$	$O(n^2)$	$O(n)$	✓ 稳定
基数排序	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	✓ 稳定

★ 常考点速记：

- 1. 平均 $O(n \log n)$ 的排序：**快排、归并、堆排** ("快归堆")
- 2. 稳定的排序：冒泡、插入、归并、计数、桶、基数 (多记「归并稳

定」)

- 3. 不稳定的排序：选择、快排、堆排 ("选快堆")
- 4. 快排最坏 $O(n^2)$ ← 有序/逆序时
- 5. 归并排序需要 $O(n)$ 额外空间

第四节 基础算法 (约 3 min)

初赛基础算法选择题——知道"是什么"即可。

算法	一句话理解	典型时间复杂度
枚举	把所有可能情况挨个试一遍	$O(n) \sim O(n^k)$
模拟	按题目规则一步一步执行	视规则而定
递推	从已知推未知，如斐波那契 $f(n)=f(n-1)+f(n-2)$	$O(n)$
贪心	每步选当前最优，不保证全局最优	$O(n \log n)$
分治	分成小问题分别解决再合并，如归并排序	$O(n \log n)$
二分	有序序列中不断折半查找	$O(\log n)$
双指针	两个指针从不同方向扫描数组	$O(n)$

★ 初赛常考：给一个题目描述，选择最适合的算法；或判断某个算法是否保证最优解（贪心常出判断题）。

长度为 $1024 = 2^{10}$ 的有序数组，二分查找最多比较 $\lceil \log_2(1024+1) \rceil = 11$ 次。重点中的重点

公式总结 ($n=1024$ 是特例展示)：

情况	次数	公式		
找到, 最少	1	1 (第一个 mid 就命中)		
找到, 最多	11	$\lfloor \log_2 n \rfloor + 1$		
找不到, 最少	10	$\lfloor \log_2(n+1) \rfloor$		
找不到, 最多	11	$\lfloor \log_2 n \rfloor + 1$		

第五节 字符数组与字符串 (约 8 min)

5.1 C 风格字符串 (`char[]`)

```

1 char s[100] = "hello";
2 cout << s;           // 输出 hello
3 cout << strlen(s);   // 5 (不包括 '\0')
4 cout << sizeof(s);   // 100 (数组大小)
5
6 char s1[] = "abc";
7 cout << sizeof (s1); // 4
8 cout << sizeof ("abc"); // 4

```

- 以 `'\0'` (空字符) 结尾
- ⚠️ `strlen` 不包含末尾的 `\0`, `sizeof` 包含整个数组大小

5.2 常用 C 字符串函数

函数	作用	示例
<code>strlen(s)</code>	求长度 (不含 <code>\0</code>)	<code>strlen("abc") = 3</code>
<code>strcmp(a, b)</code>	比较字典序: <0 / $=0$ / >0	<code>strcmp("ab", "ac") < 0</code>

函数	作用	示例
<code>strcpy(dest, src)</code>	复制字符串	<code>strcpy(s, "hi");</code>
<code>strcat(dest, src)</code>	拼接	<code>strcat(s, " world");</code>

5.3 C++ `string` 类

```
1 #include <string>
2 string s = "hello";
3 cout << s.length();           // 5
4 cout << s[0];                 // 'h'
5 cout << s.find("ll");          // 2 (未找到返回 string::npos)
6 cout << s.substr(1, 3);        // "ell" (从 1 开始, 取 3 个)
7 s += " world";                // 拼接
8
9
10 cout << sizeof (s) << endl; // 8 8也是错的, sizeof s 的结果是不固定的, 随平台而变, 和long一样, 也就是说 不要用sizeof 求 string变量
11 cout << sizeof ("hello") << endl; // 6
```

string 操作	返回值 / 作用
<code>s.length()</code> / <code>s.size()</code>	字符串长度
<code>s[i]</code>	取第 i 个字符 (0-based)
<code>s.find(t)</code>	查找子串 t 的位置, 未找到返回 <code>npos</code>
<code>s.substr(pos, len)</code>	取子串
<code>s + t</code> / <code>s += t</code>	字符串拼接

string 操作	返回值 / 作用
<code>s.compare(t)</code>	比较大小
<code>getline(cin, s)</code>	读入一整行（含空格）

★ 经典陷阱:

```

1 string s;
2 cin >> s;           // 遇到空格/换行停止
3 getline(cin, s);    // 读取一整行（包括空格）
4
5 输入:
6 a
7 bbb ccc

```

`cin >>` 和 `getline` 混用时，注意 `cin` 后的换行符会被 `getline` 读到——通常需要 `cin.ignore()`。

★ 入门真题示例:

```

1 string s = "CSP";
2 s += "-J";
3 cout << s.length();    // 输出 ? → 5

```

第六节 链表（约 8 min）

本章节多做题，根据题目学习练习

6.1 顺序表 vs 链表

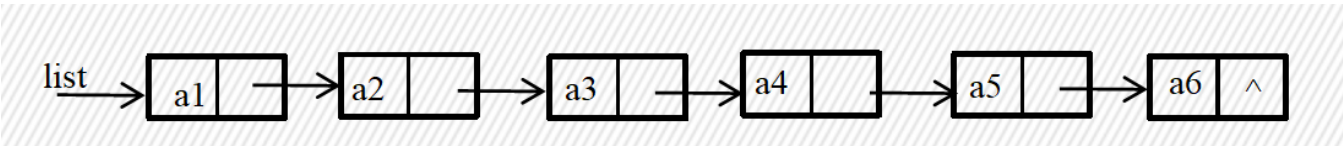


特性	顺序表 (数组)	链表
存储	连续内存	节点分散，用指针连接
随机访问	O(1)——arr[i] 直接取	O(n)——必须从头遍历
插入/删除	O(n)——要移动元素	O(1)——改指针即可 (已知位置)
空间	预先分配，可能浪费	动态分配，按需申请，自己释放
适用场景	频繁查询、数据量稳定	频繁插入删除、大小不确定

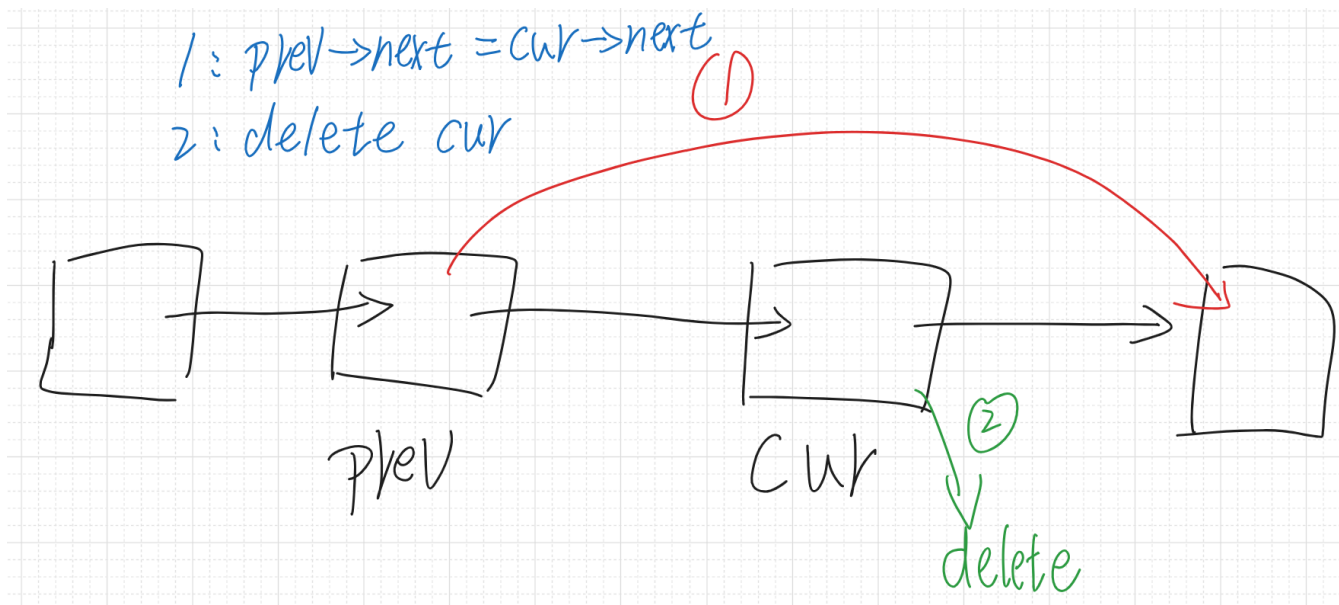
6.2 单向链表

节点结构:

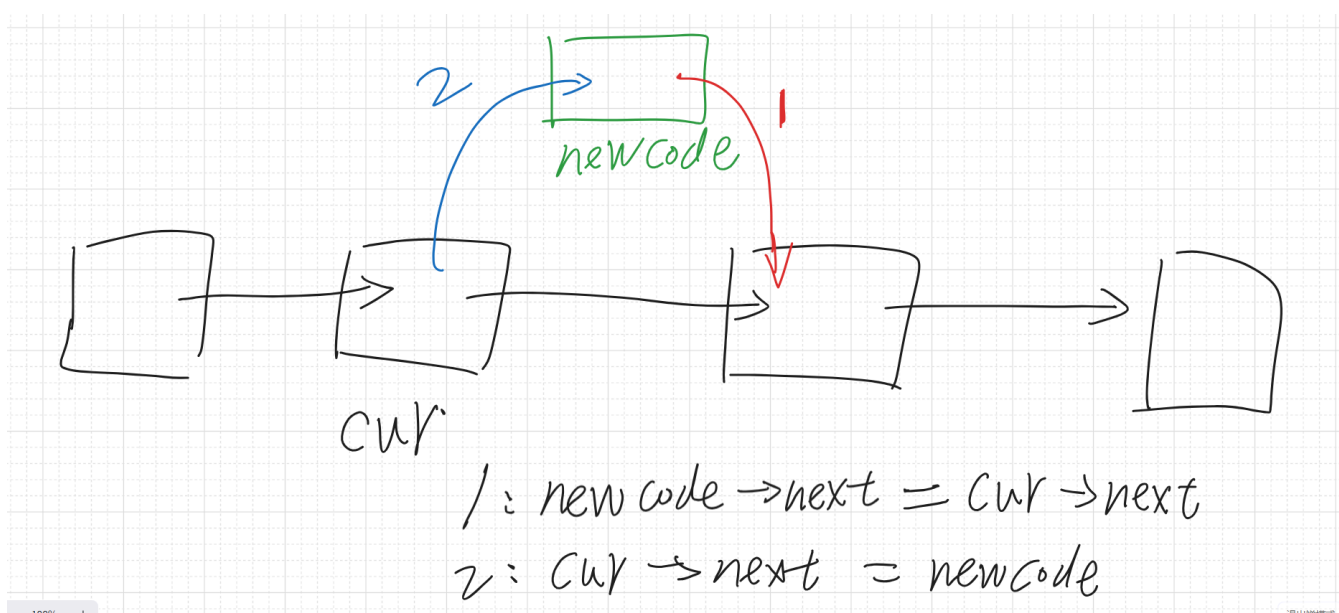
```
1 struct Node {
2     int data; // 数据域
3     Node* next; // 指针域，需要存储额外信息，所有链表的空间利用率
    是低的(低和浪费是两个概念)
4 };
```



删除结点:



新增结点:



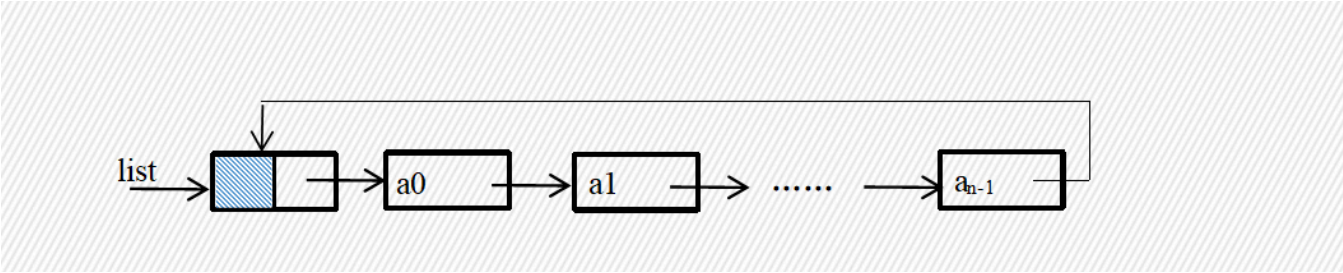
6.3 双向链表 & 循环链表

双向链表: 每个节点有 `prev` 和 `next` 两个指针, 可双向遍历。

★ 初赛常考: 给一组插入/删除操作, 问最终链表的节点顺序或某个位置的值。



循环链表：尾节点的 next 指向头节点，形成环。



第七节 栈和队列（约 10 min）

本章节多做题，根据题目学习练习

7.1 栈（Stack）—— 后进先出（LIFO）

就像一摞盘子——先放的压在最底下，后放的先拿走。

纸盒模型：只能在一个口进出，先放进去的后拿出来

核心操作：

操作	含义	时间复杂度
<code>push(x)</code>	将 x 压入栈顶	$O(1)$
<code>pop()</code>	弹出栈顶元素	$O(1)$
<code>top()</code>	获取栈顶元素（不弹出）	$O(1)$
<code>empty()</code>	判断栈是否为空	$O(1)$

- 1

栈示意图（从底到顶）：
- 2

`bottom` → [3] → [7] → [1] ← `top`
- 3

`push(9)` 之后：
- 4

`bottom` → [3] → [7] → [1] → [9] ← `top`
- 5

`pop()` 之后：
- 6

`bottom` → [3] → [7] → [1] ← `top` （9 被弹出）

数组模拟栈：

```
1 int stk[N], top = -1;
2 stk[++top] = x;      // push
3 x = stk[top--];      // pop
```

★ 常见应用：

- 1. 括号匹配 — 左括号入栈，右括号弹出检查
- 2. 表达式求值 — 中缀转后缀，后缀表达式计算
- 3. 函数调用栈 — 递归调用的底层实现

7.2 队列（Queue）—— 先进先出（FIFO）

就像排队买东西——先来的人先买到，后来的人排后面。

水管模型：一端进一端出，先进去的水，先流出来

核心操作：

操作	含义	时间复杂度
push(x)	将 x 入队（从队尾）	O(1)
pop()	弹出队头元素	O(1)
front()	获取队头元素	O(1)
back()	获取队尾元素	O(1)
empty()	判断队列是否为空	O(1)

```
1 队列示意图：
2 front → [3] → [7] → [1] ← back
3 pop() 之后：
4 front → [7] → [1] ← back    （3 被弹出）
5 push(9) 之后：
6 front → [7] → [1] → [9] ← back
```

数组模拟循环队列：

```
1 int q[N], head = 0, tail = 0;
2 q[tail++] = x;           // 入队
3 x = q[head++];          // 出队
```

⚠ 简单数组模拟会使空间无法复用，实际常用**循环队列**：`tail = (tail + 1) % N`

★ 常见应用：

1. **BFS 广度优先搜索** — 逐层遍历
2. **树的层序遍历**
3. **缓冲区 / 消息队列**

7.3 栈 vs 队列

	栈	队列
原则	后进先出 (LIFO)	先进先出 (FIFO)
比喻	一摞盘子、盒子	排队、水管
操作端	仅栈顶	队头出、队尾入
典型应用	括号匹配、递归	BFS、打印队列

第八节 STL

- vector
- map
- set

了解语法



课堂速记要点

1. 算法五特征：有穷、确定、可行、输入、输出
 2. 复杂度排序： $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
 3. 稳定排序：冒泡、插入、归并、计数、基数；不稳定：选快堆
 4. 平均 $O(n \log n)$ 的排序：快归堆
 5. 值传递不影响实参，引用传递影响
 6. 指针 `*p` 解引用，`&x` 取地址
 7. 类中 `private` 外部不可直接访问
 8. `strlen` 不含 `\0`，`sizeof` 含整个数组
 9. 链表插入/删除 $O(1)$ ，随机访问 $O(n)$ ；数组相反
 10. 栈 LIFO（括号匹配），队列 FIFO（BFS）
-



速查答案记忆卡

1	算法五特征	└→ 有穷、确定、可行、输入、输出
2	复杂度等级	└→ $1 < \log n < n < n \log n < n^2 < 2^n$
3	稳定排序	└→ 冒泡 插入 归并 计数 基数
4	不稳定排序	└→ 选择 快排 堆排（选快堆）
5	$O(n \log n)$	└→ 快排 归并 堆排（快归堆）
6	链表	└→ 插入删除 $O(1)$ 随机访问 $O(n)$ 用数组模拟
7	栈	└→ 后进先出 push/pop/top
8	队列	└→ 先进先出 push/pop/front/back
9	<code>strlen(s)</code>	└→ 长度不含 <code>\0</code>
10	<code>sizeof(s)</code>	└→ 数组分配大小